UNIT-I


COMMON
ARCHITECTURAL
PRINCIPLES
of
DEEP LEARNING
Building Blocks Behind Powerful Models

Topics :

- Common Architectural Principles of Deep Networks
  - Parameters
    - 1. Weights
    - 2. Bias
  - Layers
  - Activation Functions
  - Loss Functions
  - Hyper Parameters

Common Architectural Principles of DL

- The **Common Architectural Principles of Deep Learning** are the **fundamental building blocks** and **design concepts** used to **construct deep neural networks**.
- These principles help **deep learning models learn complex patterns from data** and **achieve high accuracy in tasks** such as **image classification, speech recognition, natural language processing, and medical diagnosis**.
- The **architecture of a deep neural network** is based on several common principles such as **parameters, layers, activation functions, loss functions, and hyperparameters**. These components work together to enable the network to **learn complex patterns and make accurate predictions.**

1. Parameters

- Parameters are the **learnable values** of a **neural network**. **During training**, **the network automatically adjusts these values to minimize prediction error**.
- The Two Main Parameters are:
  1. **Weights(W)**
  2. **Bias(b)**

1. **Weights(W):** **Weights are learnable numerical parameters** in a **neural network** that determine the **importance of each input feature**.
   - They are assigned **each input in a neural network**.
   - They determine how important or influential each input feature is in making a prediction.
   - **During training,** the **network automatically learns the best weight values** by **minimizing the prediction error** and **improve the model's accuracy.**

WEIGHTS – EXAMPLE

Weights determine the importance of each input feature.

INPUT FEATURES
(Independent Variables)



Study Hours (x_1) = 8



Attendance (%) (x_2) = 90



Mobile Usage (hrs/day) (x_3) = 5

WEIGHTS (Importance)

w_1 = 0.8

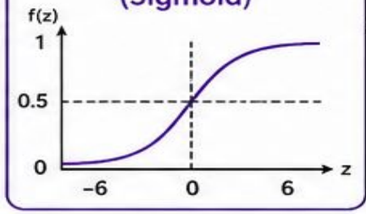
w_2 = 0.5

w_3 = −0.4

Bias (b) = 1

Weighted Sum (z) = 50.4

Activation Function (Sigmoid)



Prediction

High Marks

(Probability ≈ 0.99)

INTERPRETATION OF WEIGHTS

- 0.8 → Strong positive impact
- 0.5 → Positive impact
- −0.4 → Negative impact
- 0 → No impact

Input Feature	Value (x)	Weight (w)	Contribution (x × w)
Study Hours (x_1)	8	0.8	8 × 0.8 = 6.4
Attendance (x_2)	90	0.5	90 × 0.5 = 45.0
Mobile Usage (x_3)	5	−0.4	5 × (−0.4) = −2.0
Bias (b)	1	–	= 1.0
Total Weighted Sum (z)			= 50.4

WEIGHTS FORMULA (GENERAL)

For n input features:

$$z = \sum_{i=1}^{n} w_i x_i + b$$

- x_i → i^{th} input feature
- w_i → weight of ith input
- b → bias
- z → weighted sum (input to activation function)

For this example (3 inputs):

$$z = (x_1 \times w_1) + (x_2 \times w_2) + (x_3 \times w_3) + b$$

$$z = (8 \times 0.8) + (90 \times 0.5) + (5 \times (-0.4)) + 1$$

$$z = 6.4 + 45.0 - 2.0 + 1$$

$$z = 50.4$$

MEANING OF WEIGHTS

- Higher positive weight → increases the output
- Negative weight → decreases the output
- Weight close to zero → very little effect
- Weights are learned automatically during training to reduce error.



Weights tell the model what is important!



In Deep Learning, weights are NOT fixed. They are updated during training using Backpropagation and Gradient Descent to minimize the prediction error and improve accuracy.

2. Bias(b): Bias is a **learnable parameter** in a neural network that is **added to the weighted sum of inputs before applying the activation function**.

- It allows the model **to shift the activation function** and **produce accurate predictions even when all input values are zero.**
- **In simple words, Bias helps the neuron adjust its output independently of the input values.**

Bias shifts the activation function.

Without bias,

$$z = Wx$$

With bias,

$$z = Wx + b$$

Bias allows better fitting of data.

BIAS IN NEURAL NETWORKS – EASY TO UNDERSTAND

1. WHAT IS BIAS?

Bias (b) is a learnable parameter that is added to the weighted sum of inputs before applying the activation function.

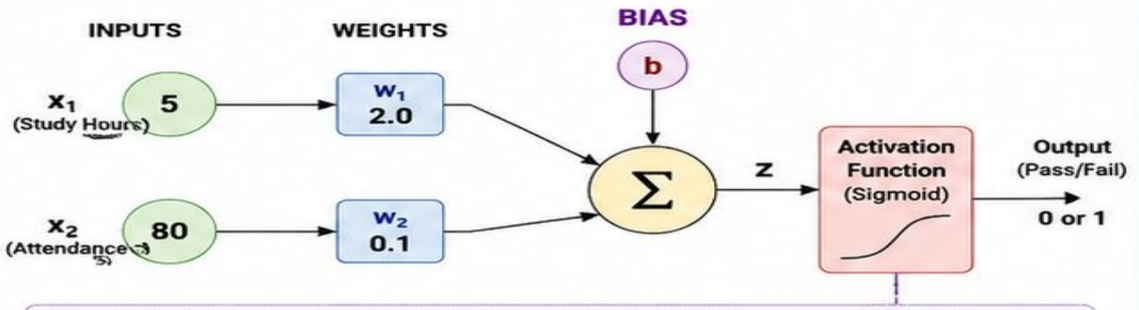
It acts as a starting point or an adjustment. It shifts the output of the neuron left or right and helps the model make better decisions.

Formula:

$$z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$$

Weighted Sum (Inputs × Weights) — Bias (Adjustment)

2. HOW BIAS WORKS IN A NEURON



Step 1: Weighted Sum

$z_1 = (x_1 \times w_1) + (x_2 \times w_2)$
$= (5 \times 2.0) + (80 \times 0.1)$
$= 10 + 8 = 18$

Step 2: Add Bias

$z = z_1 + b = 18 + b$

Step 3: Apply Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

(Output between 0 and 1)

3. SEE THE EFFECT OF BIAS

Case	Bias (b)	Calculation z = 18 + b	Interpretation
Case 1 (No Bias)	0	18 + 0 = 18	Normal decision
Case 2 (Negative Bias)	−15	18 − 15 = 3	Score becomes smaller → Model is stricter
Case 3 (Positive Bias)	+10	18 + 10 = 28	Score becomes larger → Model is more lenient

Bias shifts the output.
Positive bias → increases the output.
Negative bias → decreases the output.

4. REAL-LIFE EXAMPLES TO UNDERSTAND BIAS

A. CRICKET SELECTION ANALOGY

A player is selected if score ≥ 50 runs.

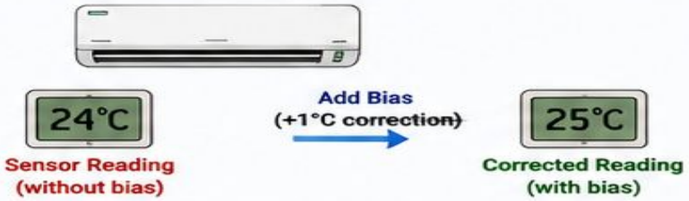


Player Runs	Bias	Adjusted Score	Selected?
48	0 (No Bias)	48	✗ Not Selected
48	+5	48 + 5 = 53	✓ Selected

Bias gives an extra adjustment to help the final decision.

B. AIR CONDITIONER ANALOGY

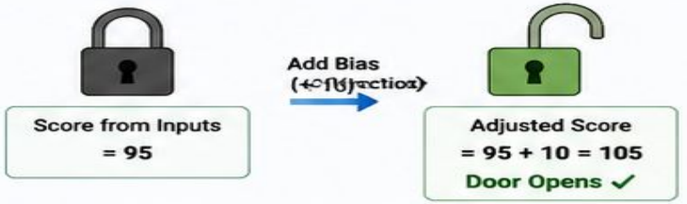
Actual Room Temp = 25°C, but sensor reads 24°C (1°C low)



Bias corrects the system and gives accurate output.

C. DOOR LOCK ANALOGY

Door opens only if password score ≥ 100.



Bias shifts the decision threshold.

5. WHY DO WE NEED BIAS?

If all inputs are zero:

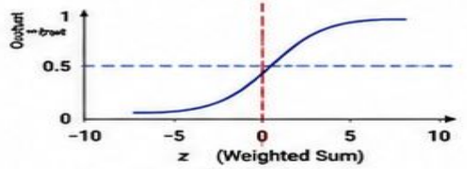
$x_1 = 0,\ x_2 = 0$

Without Bias (b = 0)
z = 0 (always)
Neuron can not learn different outputs.

With Bias (b = 5)
z = 0 + 5 = 5
Neuron can still produce a useful output.

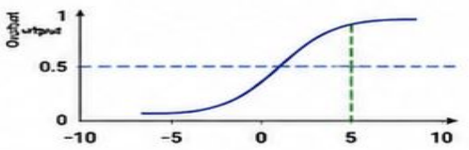
6. VISUAL EFFECT OF BIAS ON ACTIVATION FUNCTION

Without Bias (b = 0)



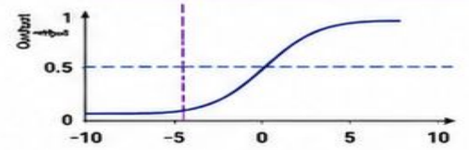
Decision boundary at z = 0

With Positive Bias (b = +5)



Curve shifts left → easier to get high output

With Negative Bias (b = −5)



Curve shifts right → harder to get high output

7. EASY WAY TO REMEMBER

Weights tell the model **HOW IMPORTANT EACH INPUT IS.**

Bias tells the model **WHERE THE DECISION STARTS.**

8. ONE LINE DEFINITION (FOR EXAM)

Bias is a learnable parameter added to the weighted sum of inputs. It shifts the activation function and enables the neural network to make accurate predictions even when the input values are small or zero.

9. SUMMARY

- **Weights** decide the importance of inputs.
- Bias gives a starting point or adjustment.
- Formula: Output = (Inputs × Weights) + Bias
- Bias helps the model be flexible and accurate.

2. Layers

The mother art is architecture. Without an architecture of our own we have no soul of our own civilization.

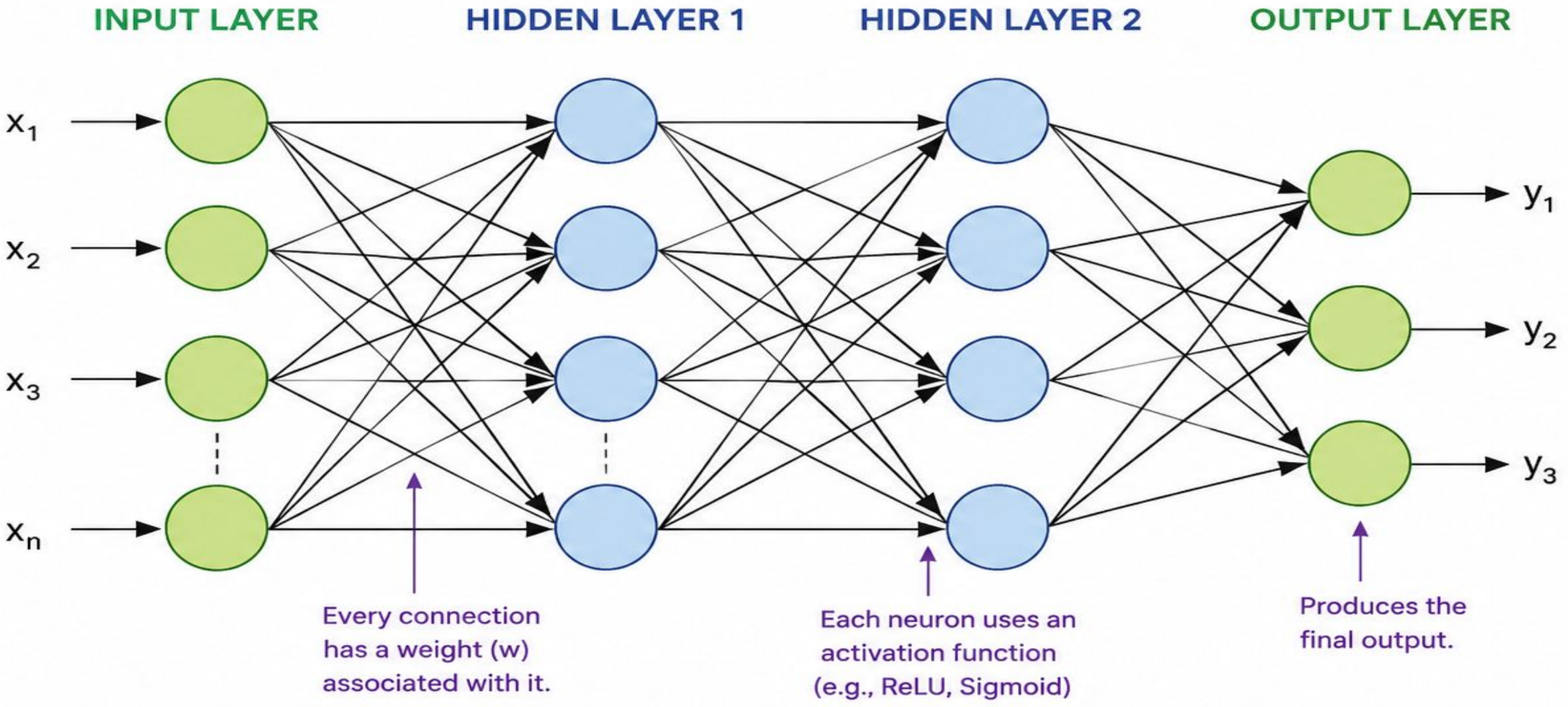
-Frank Lloyd Wright

- **A Layer is a collection of neurons** that **performs a specific operation on the input data**.
- **Neural networks** are **organized into multiple layers**, where **each layer processes the information received from the previous layer** and **passes its output to the next layer**.
- The **arrangement of layers enables the network** to **learn simple patterns in the initial layers** and **more complex patterns in the deeper layers**.

A **typical neural network** consists of **three main types of layers**:

1. **Input Layer**
2. **Hidden Layer(s)**
3. **Output Layer**

ARTIFICIAL NEURAL NETWORK (ANN)



How It Works:

1. Inputs are fed into the input layer.
2. Each connection multiplies the input by a weight.
3. The neuron sums the weighted inputs, applies an activation function.
4. Output is passed to the next layer until the final output is produced.

Input Neuron Hidden Neuron Output Neuron Weighted Connection

2. LAYERS

1. **Input Layer:** The **Input Layer** is the **first layer of a neural network.**

- **It receives the input data from the outside world** and **forwards it to the hidden layer without performing any mathematical computations.**
- The **number of neurons in the input layer** is **equal to the number of input features.**

Characteristics:

1. **First layer of the network**
2. **Receives input data**
3. **No calculations are performed**
4. **Passes data to the hidden layer**

Mathematical Representation

Suppose the input vector is

$$X = [x_1, x_2, x_3, \ldots, x_n]$$

where

- $x_1, x_2, \ldots, x_n$ are the input features.

The input layer simply forwards these values.

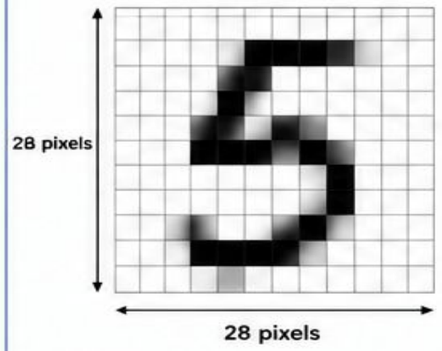
INPUT LAYER – EXAMPLE

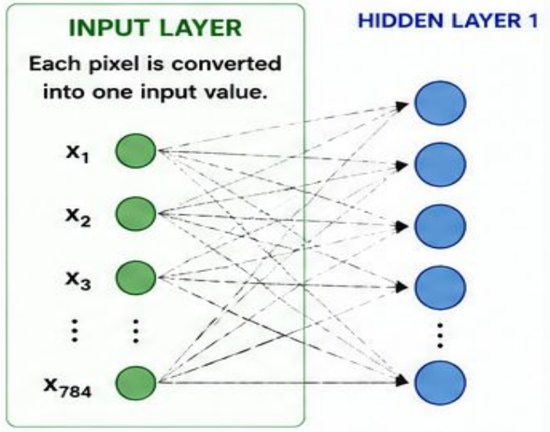
The **Input Layer** is the first layer of a neural network. It receives the input features from the outside world and passes them to the hidden layer without any computation.

1. IMAGE CLASSIFICATION EXAMPLE

Example: Digit Recognition

Input Image (28 × 28)



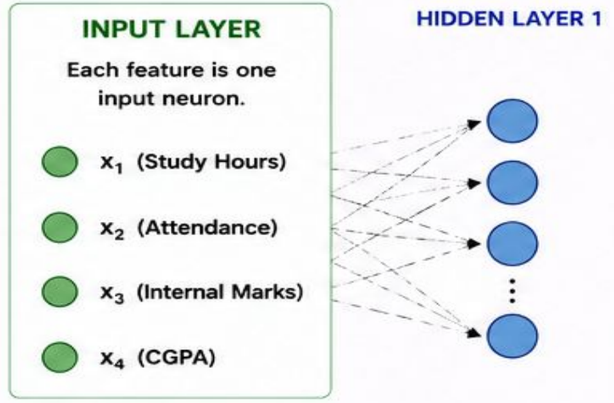


Number of Input Neurons
= 28 × 28 = **784**

2. TABULAR DATA EXAMPLE

Example: Student Performance Prediction

Feature	Value
Study Hours	6
Attendance (%)	85
Internal Marks	78
Previous CGPA	8.2



Number of Input Neurons
= Number of Features = **4**

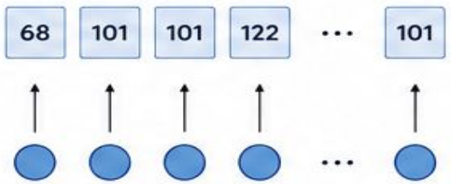
KEY POINTS ABOUT INPUT LAYER

- Receives raw input data.
- Performs no computation.
- Number of neurons = Number of input features.
- Passes data to the hidden layer.
- Acts as a bridge between outside world and the network.

OTHER EXAMPLES

Example: Text Classification

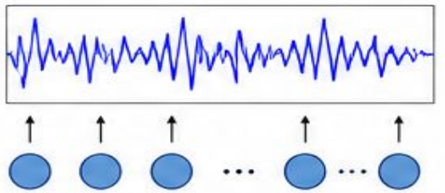
Input: "Deep Learning is awesome"



Each number (ASCII / Token ID) is one input neuron

Example: Audio Signal

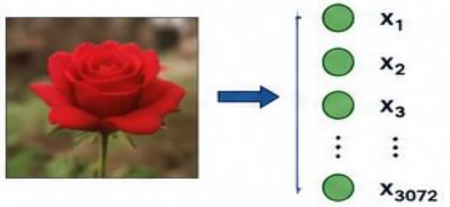
Input: Sound Wave



Each sample point is one input neuron

Example: Color Image (RGB)

Input Image (32 × 32 × 3)



Number of Input Neurons
= 32 × 32 × 3 = **3072**

In all cases, the input layer only accepts data and forwards it. It does not change the data.

2. Layers

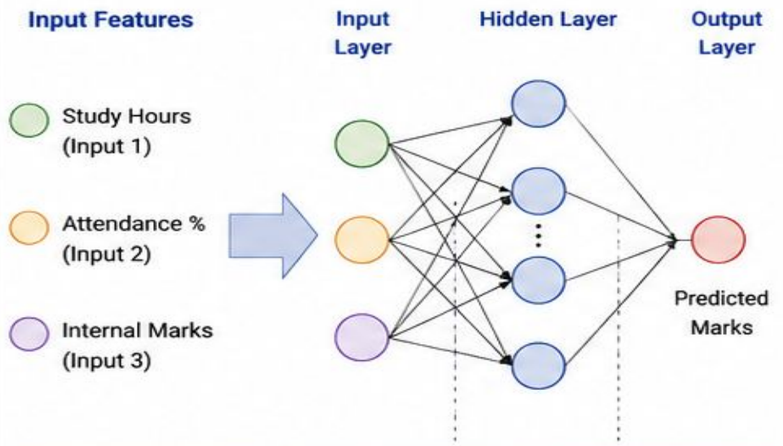
2. Hidden Layer: The **Hidden Layer** is the **computational layer of a neural network**.

- It **receives data from the input layer**, **performs mathematical operations**, **applies activation functions,** and **extracts useful features.**
- **Hidden layers are called "hidden"** because **their outputs are not directly visible to the user.**

- **Without hidden layers**, the **network can only solve simple linear problems.**
- **Hidden layers enable the network** to **solve complex, non-linear problems.**

INPUT LAYER AND HIDDEN LAYER – EXAMPLES

Example 1: Student Marks Prediction



Number of Input Neurons = 3 (One for each feature)

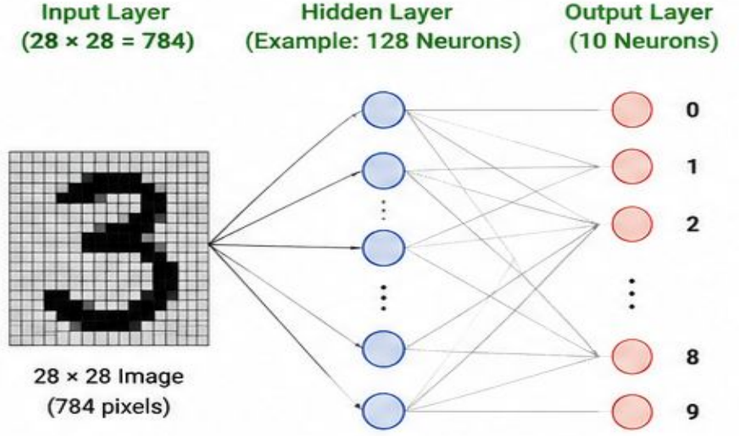
Number of Hidden Neurons = 4 (Extract important patterns)

Number of Output Neurons = 1 (Predicted Marks)

How it works:

- Input layer receives the student data.
- **Hidden layer** learns complex patterns and relationships.
- Output layer gives the final predicted marks.

Example 2: Image Recognition (Digit Classification)



Number of Input Neurons = 784 (One for each pixel)

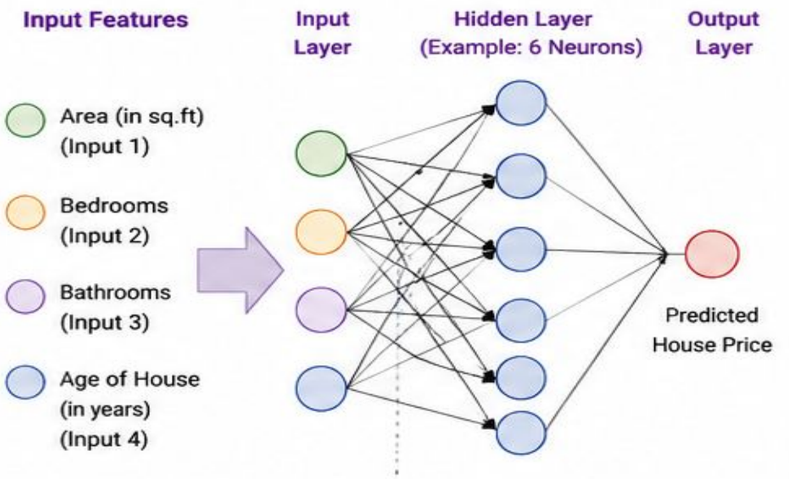
Number of Hidden Neurons = 128 (Learns edges, shapes, patterns)

Number of Output Neurons = 10 (Classes 0 to 9)

How it works:

- Each pixel goes to the **input layer (784 neurons)**.
- **Hidden layer detects patterns like edges, strokes, shapes.**
- Output layer tells **which digit it is (0 to 9)**.

Example 3: House Price Prediction



Number of Input Neurons = 4 (One for each feature)

Number of Hidden Neurons = 6 (Learns complex relationships)

Number of Output Neurons = 1 (Predicted Price)

How it works:

- Input layer takes house features.
- **Hidden layer** learns important combinations.
- Output layer predicts the final house price.

Key Idea:

The input layer receives raw data.

The hidden layer processes the data, learns patterns and relationships.

The output layer produces the final prediction.

Summary Table

Example	Input Features	Number of Input Neurons	Hidden Layer (Example)	Number of Hidden Neurons	Output Layer (Example)	Number of Output Neurons
1. Student Marks Prediction	Study Hours, Attendance, Internal Marks	3	4 Neurons	4	Predicted Marks	1
2. Image Recognition (Digit Classification)	28 × 28 Pixels = 784	784	128 Neurons	128	Digits (0–9)	10
3. House Price Prediction	Area, Bedrooms, Bathrooms, Age	4	6 Neurons	6	Predicted Price	1

2. Layers

3. Ouput Layer: The **Output Layer** is the final **layer of a neural network**. It receives processed information **from the hidden layer and produces the final prediction or decision**.

Characteristics

- Last layer
- Produces predictions
- Uses activation functions like Sigmoid or Softmax
- Number of neurons depends on the task

Types of Output Layer:

1. Binary Classification

Predicts one of two classes.

Examples

- ✔ Spam or Not Spam
- ✔ Disease or No Disease
- ✔ Cat or Not Cat

2. Multi-Class Classification

Predicts one class among many.

Example

Digit Recognition

Classes

0–9

EXAMPLE: IDENTIFYING A CAT IN A PICTURE

A deep neural network learns hierarchical features from the image and predicts “Cat”.

INPUT LAYER

Receives the image pixels.



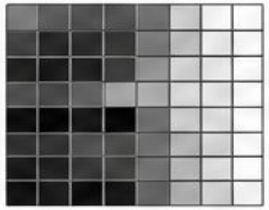
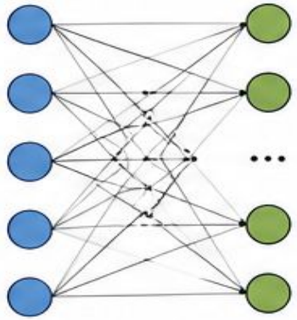


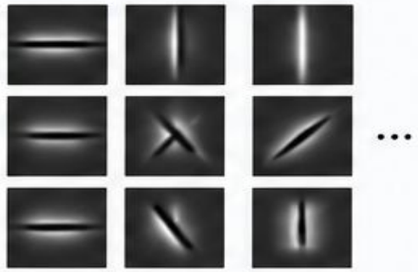
Image Pixels
(R, G, B values)

HIDDEN LAYER 1

Detects edges and lines.

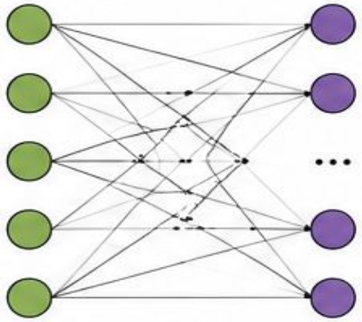


Learned Features
(Edges & Lines)



HIDDEN LAYER 2

Detects eyes, ears, and whiskers.

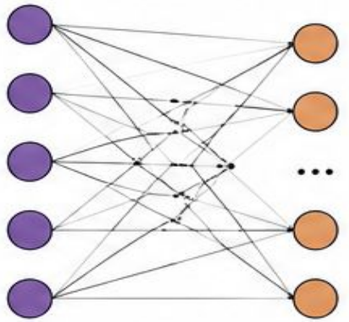


Learned Features
(Eyes, Ears, Whiskers)



HIDDEN LAYER 3

Combines features to recognize the whole cat.



Learned Features
(Parts → Whole Cat)



OUTPUT LAYER

Predicts “Cat” with the highest probability.

Cat	0.97
Dog	0.02
Rabbit	0.005
Bird	0.003

PREDICTION:



CAT

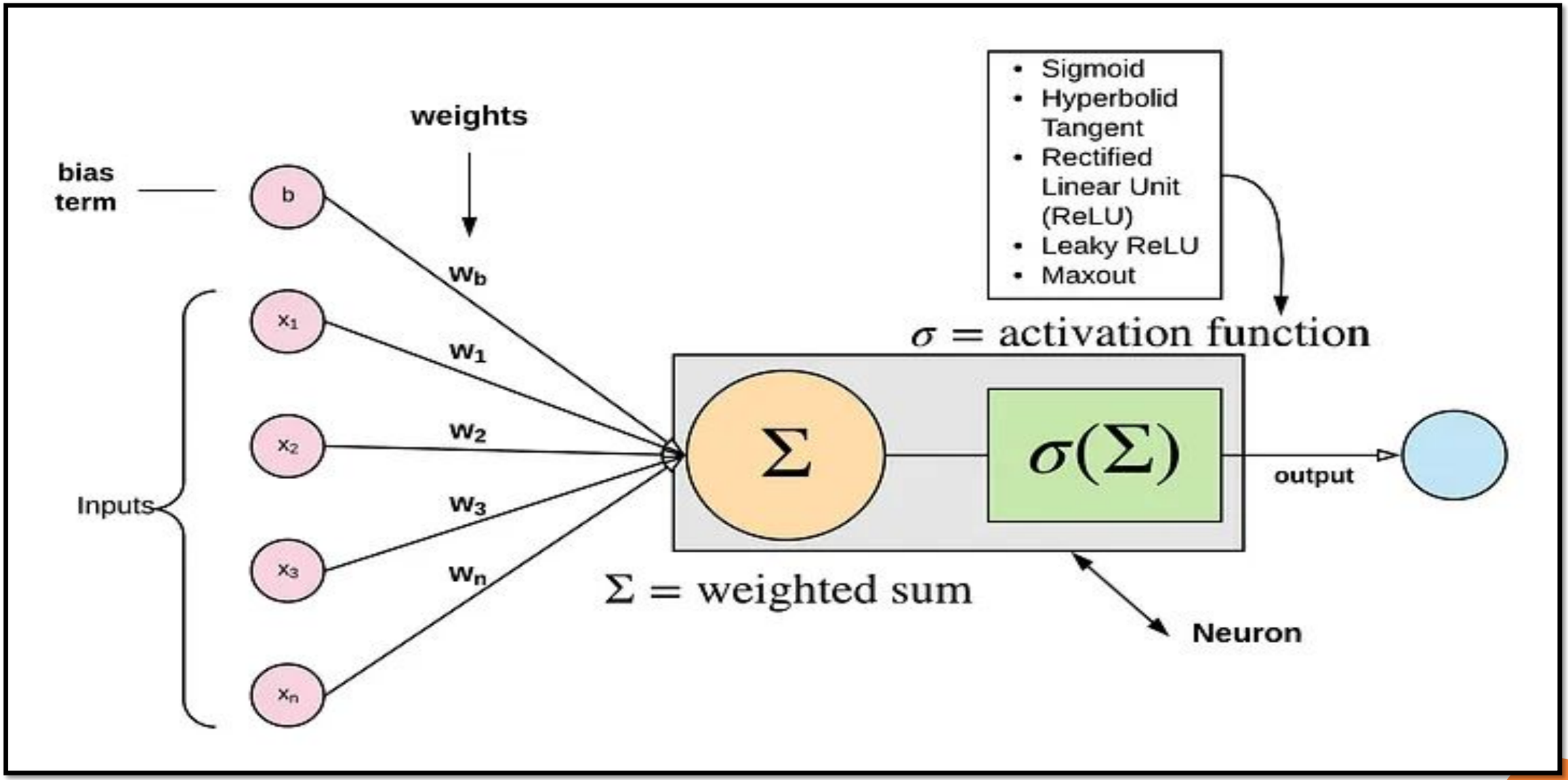
HOW IT WORKS

1. Input Layer takes the raw pixel values of the image.
2. Hidden Layer 1 detects basic patterns like edges and lines.
3. Hidden Layer 2 detects important parts like eyes, ears, whiskers.
4. Hidden Layer 3 combines these parts to understand the whole cat.
5. Output Layer predicts “Cat” with the highest probability.

3. Activation Functions

- In the **neural network** **activation function** plays an important role.
- The activation function determines the **output of a deep learning network.**
- An activation function is a **mathematical operation** applied to the **output of a neuron** in a neural network.

  It determines **whether a neuron should be activated or not**, based on the **weighted sum of its inputs.**
- An activation function is **an internal state of a neuron** that converts an **input signal to an output signal**.
- The **primary role of the Activation Function** is to **transform the summed weighted input from the node into an output value** to be fed to the **next hidden layer or as output.**

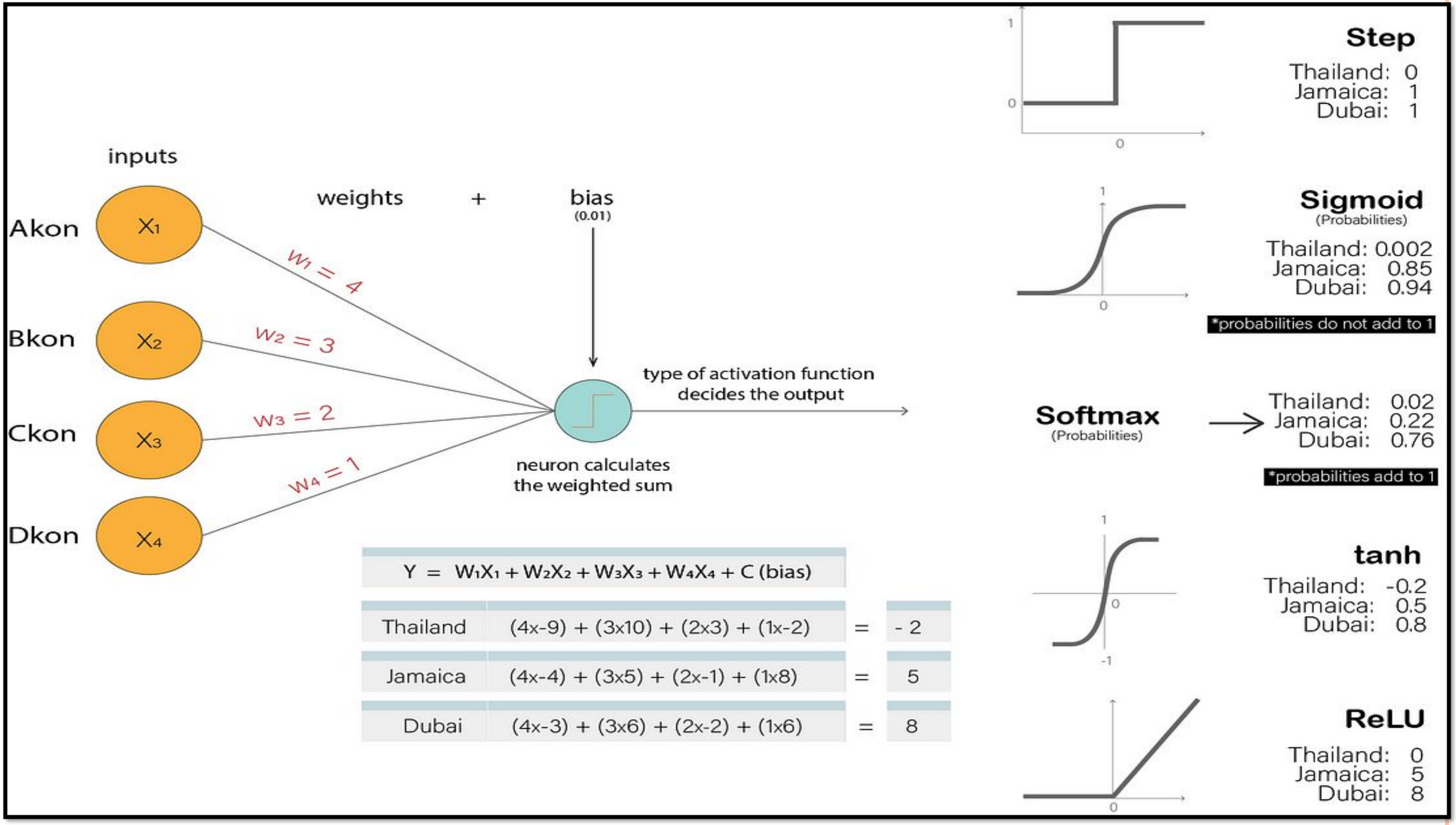

weights
bias term
b
w_b
x_1
w_1
x_2
w_2
Inputs
w_3
x_3
w_n
x_n
• Sigmoid
• Hyperbolid Tangent
• Rectified Linear Unit (ReLU)
• Leaky ReLU
• Maxout
σ = activation function
Σ
$\sigma(\Sigma)$
output
Σ = weighted sum
Neuron

- Activation functions provide **non-linear properties** to the **neural network**. **Without the activation function**, the **output values from the neurons** can range between **(- infinity) to (+infinity).**
- **For Eg:** A **group of 4 friends is deciding to go on a vacation**. They have narrowed down their options to **three countries**. **Thailand, Jamaica, and Dubai**.
- In order to chose the **final destination** they **each decide to vote for the countries** on **a scale of -10 to 10**. **-10 being least interested** and **10 being most interested**. **0 can be considered as neutral.**
- From the **table below**, you can notice, **Bkon** for e.g. is **most interested in Thailand** and **least interested in visiting Jamaica.**

Votes for vacation destinations

	Akon	Bkon	Ckon	Dkon
Thailand	-9	10	3	-2
Jamaica	-4	5	-1	8
Dubai	-3	6	-2	6

- Let us assign **initial random weights** to **each variable.** For the **sake of simplicity in calculations**, we will keep the **bias value = 0.01 or simply 0.**
- **Observe the figure below:** The neuron calculates the **weighted sum of its inputs** and **provides and output** value.
- The **neuron here**, **without an activation function** would **provide 1 value** each for **Thailand, Jamaica, and Dubai**. i.e. **Thailand = -2**, **Jamaica = 5**, and **Dubai = 8**.


inputs
weights
+
bias
(0.01)
Akon
Bkon
Ckon
Dkon
X1
X2
X3
X4
W1 = 4
W2 = 3
W3 = 2
W4 = 1
type of activation function
decides the output
neuron calculates
the weighted sum
Y = W1X1 + W2X2 + W3X3 + W4X4 + C (bias)
Thailand (4x-9) + (3x10) + (2x3) + (1x-2) = - 2
Jamaica (4x-4) + (3x5) + (2x-1) + (1x8) = 5
Dubai (4x-3) + (3x6) + (2x-2) + (1x6) = 8
Step
Thailand: 0
Jamaica: 1
Dubai: 1
Sigmoid
(Probabilities)
Thailand: 0.002
Jamaica: 0.85
Dubai: 0.94
*probabilities do not add to 1
Softmax
(Probabilities)
Thailand: 0.02
Jamaica: 0.22
Dubai: 0.76
*probabilities add to 1
tanh
Thailand: -0.2
Jamaica: 0.5
Dubai: 0.8
ReLU
Thailand: 0
Jamaica: 5
Dubai: 8

- There are **3 types of Activation Functions**
  - **1) Binary Step Functions**
  - **2) Linear Activation Function**
  - **3) Non Linear Activation Functions**

- **1) Binary Step Function:** Binary step function is one of the **simplest activation functions.** The function produces **binary output** and thus the **name *binary step function*.** This activation function can be used for **binary classifications.**

- It produces **binary output as 0 or 1. Any value below 0 is automatically given a value of 0** and **any value above 0 is given a value of 1.**
- If the **input is greater than zero, turn/send a pulse to activate (ON), otherwise (OFF).**
- Binary Step Function depends on the **Threshold Value**, that decides **whether the neuron should be active or not.**

Limitations: It cannot provide Multi Value Outputs.

- **i.e, it cannot be used in Multi Class Classification Problems.**

2. Linear Activation Function:

The linear activation function, also known as **"no activation,"** or **"identity function",** is one of the **most straightforward activation functions**, where the **output is identical to the input**.

- **Equation :** Linear function has the equation similar to as of a straight line i.e. **y = x**
- **Range :** -inf to +inf
- The **linear activation function** typically **adds the weighted sum with bias and passes it as output.**

Limitations: It cannot provide Multi Value Outputs.

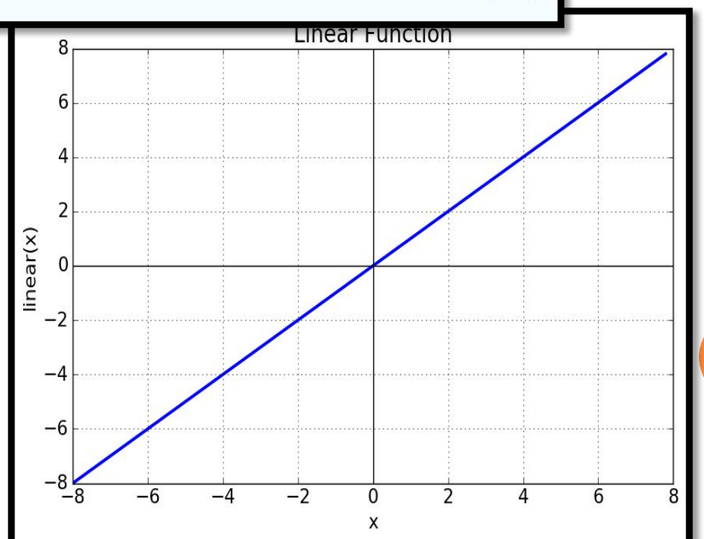
i.e, it cannot be used in Multi Class Classification Problems.

Mathematically, it can be defined as:

$$f(x) = x$$

where :

$$x = \sum Input * Weight + Bias(b)$$



3. Non Linear Activation Function: A **nonlinear function** used to **send the output signal either on or off** to **a neuron** is known as an **activation function.**

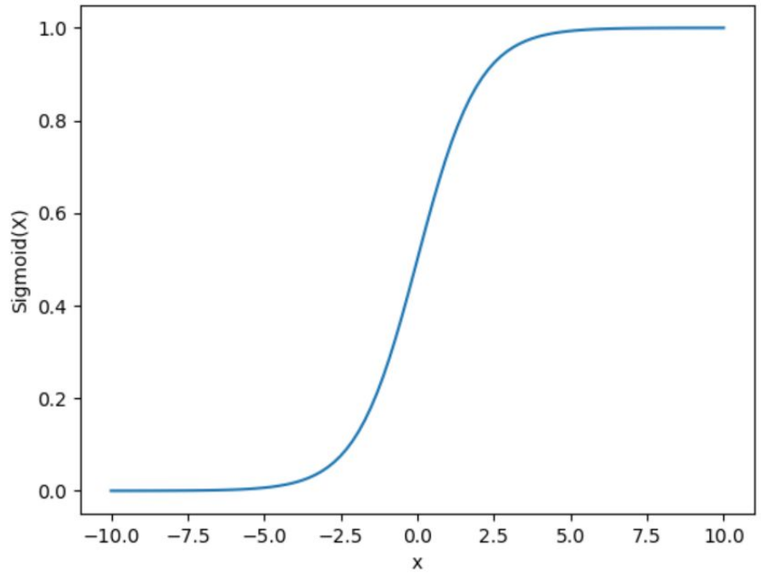
There are different types of Non Linear Activation Functions:

- ✔(a) Sigmoid Activation Function
- ✔(b) Tanh Activation Function
- ✔(c) ReLU Activation Function
  - ✔**Leaky ReLu Activation Function**
- ✔(d) Softmax Activation Function

A)Sigmoid Activation Function: Introduced in the early 1990s, This function is a probabilistic approach towards decision making and output range is between 0 and 1.

- Since the **range is minimum** , **predictions (decisions)** would be **more accurate.**
- The Sigmoid function, often represented as **(x),** features an **S-shaped curve.**

$$f(x) = \frac{1}{1 + e^{-x}}$$



The sigmoid function, also known as the **squashing function**, takes the **input from the previously hidden layer** and **squeezes it between 0 and 1.** Such as **"yes" or "no", or 1 or 0.** The sigmoid function can **output a probability between 0 and 1,** which can be used to predict the likelihood of a particular class.

Applications:

1. **Binary Classification Problems:**

We can use the sigmoid function in binary classification problems as it returns the **output between 0 and 1.**

2. Image Datasets and Neural Networks:

The sigmoid function can be used for **neural networks** on **image datasets** for performing tasks like **image segmentations, classifications, etc.**

Neural network architectures and components like **activation functions** enable **modeling of complex non-linear patterns** across domains like:

- **Image recognition**
- **Natural language processing**
- **Speech recognition**
- **Reinforcement learning**

Drawback of the Sigmoid Function: One of the significant issues with the sigmoid is the **lack of weight updating.** Therefore, the function sometimes returns **small values as outputs**, making **no changes in the weights and biases** that cause the **vanishing gradient problem.**

- This problem arises in **really large Deep Neural Networks.**
- This leads to **more computational costs and poor learning**



B) Hyperbolic Tangent Activation Function (Tanh)

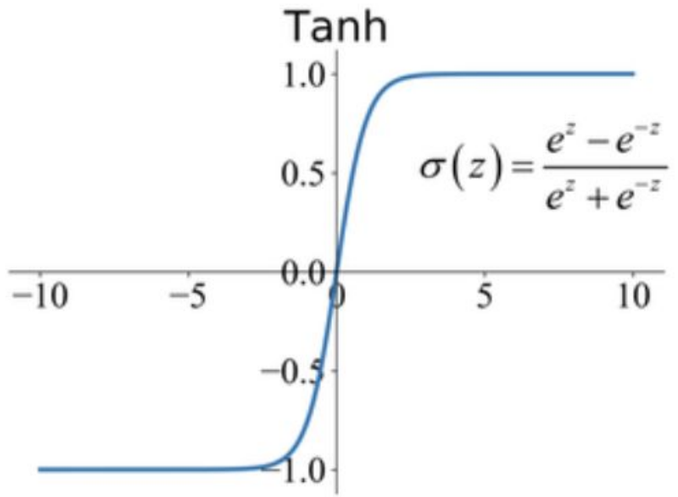
Used widely in 1990's-2000's, it overcomes the disadvantage of the **sigmoid activation function** by ***extending the range to include -1 to 1***.

It is also having the same **S-shape** with the difference in **output range of -1 to 1.**

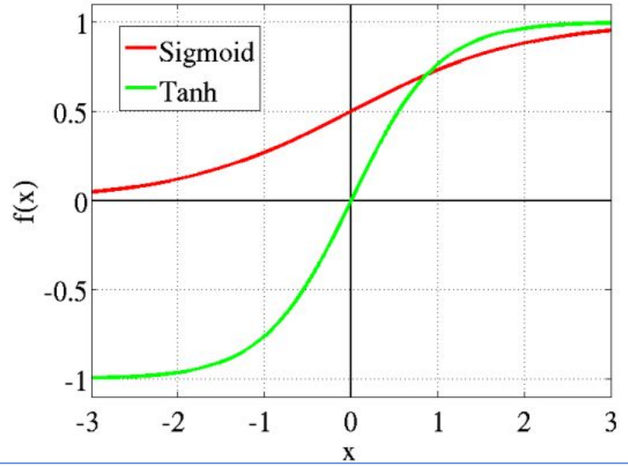
In Tanh, the **larger the input (more positive),** the closer the **output value will be to 1.0**, whereas the **smaller the input (more negative),** the closer the **output will be to -1.0**.

$$Tanh$$

$$f(x) = \frac{\left(e^{x} - e^{-x}\right)}{\left(e^{x} + e^{-x}\right)}$$



But it did not solve the **vanishing gradient problem**.



Advantages of using this activation function are:

- The output of the tanh activation function **is Zero centered**; hence we can easily map the output values as **strongly negative, neutral, or strongly positive.**
- Usually **used in hidden layers of a neural network** as its **values lie between -1** to; therefore, the mean for the hidden layer comes out to be 0 or very close to it. It helps in **centering the data and makes learning for the next layer much easier.**

Comparision of Sigmoid and Tanh Activation Function:

- Finally, we'll present an example of applying these activation functions in a **simple neuron of two input features and weights** . Therefore, we can see the **output value and the gradient** when we use the **sigmoid (left) and the tanh (right) activation function**.

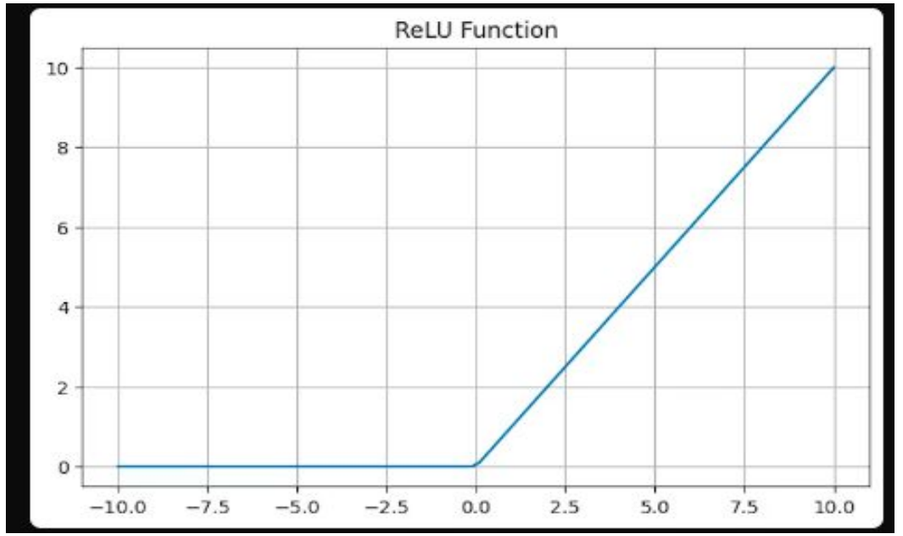
C) ReLU Activation Function: ReLU is nothing but **Rectified Linear Unit**.

ReLU is a piecewise linear function that results in the **input directly if it is positive**. Otherwise, **it outputs zero.**

Equation :- It gives an output x if x is positive and 0 otherwise.

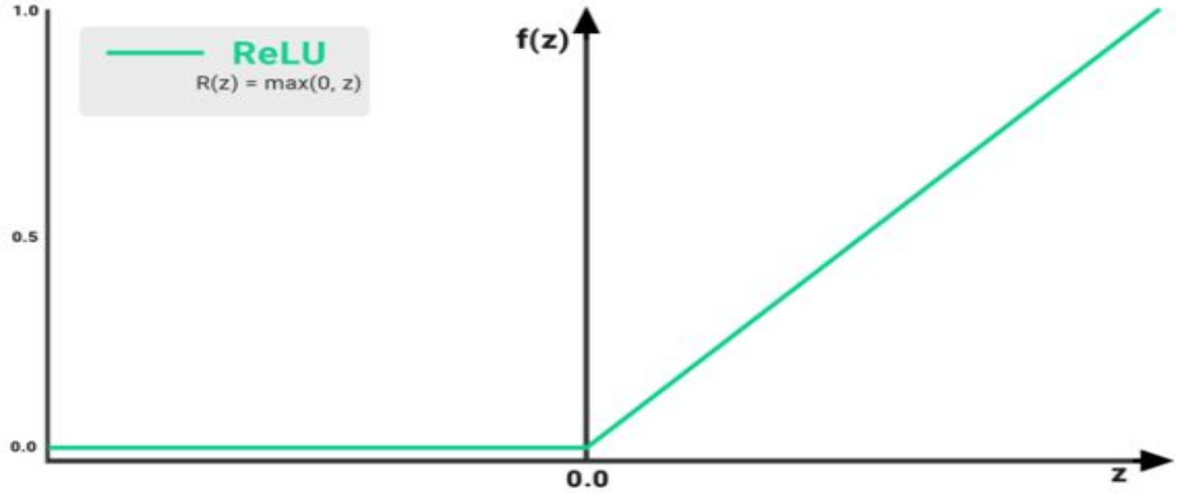
ReLU is most commonly used in **Large Deep Neural Networks** with **lots and lots of hidden layers**. Since ReLU only **requires some threshold operations**, it is computationally **efficient** even though the **dataset and the network are large.**

$$f(x) = \max(0, x)$$



Also, ReLU is a great choice to **avoid the Vanishing Gradient Problem**

- This is the most frequently used activation unit in deep learning. **R(x) = max(0, x) . Thereby, if x < 0, R(x) = 0 and if x ≥ 0, R(x) = x.**



Function	Equation	Range	Derivative
ReLu (Rectified Linear Unit)	$f(x) = \begin{cases} 0 \text{ for } x < 0 \\ x \text{ for } x \geq 0 \end{cases}$	$0, +\infty$	$f'(x) = \begin{cases} 0 \text{ for } x < 0 \\ 1 \text{ for } x \geq 0 \end{cases}$

For the **negative input values**, the **result is zero**, that means the **neuron does not get activated**. Since only a certain number of neurons are activated, the ReLU function is far more computationally efficient when **compared to the sigmoid and tanh function**.

- ReLU is used as a **middle-layer (either convolution or dense) activation function** because it is a **non-linearity that works well and is robust to the vanishing gradient problem** (**as opposed to tanh or sigmoid**).
- This means that the **neurons will only be deactivated** if the **output of the linear transformation is less than 0.**

Use Cases: Many types of neural networks **use ReLU as their default activation function**

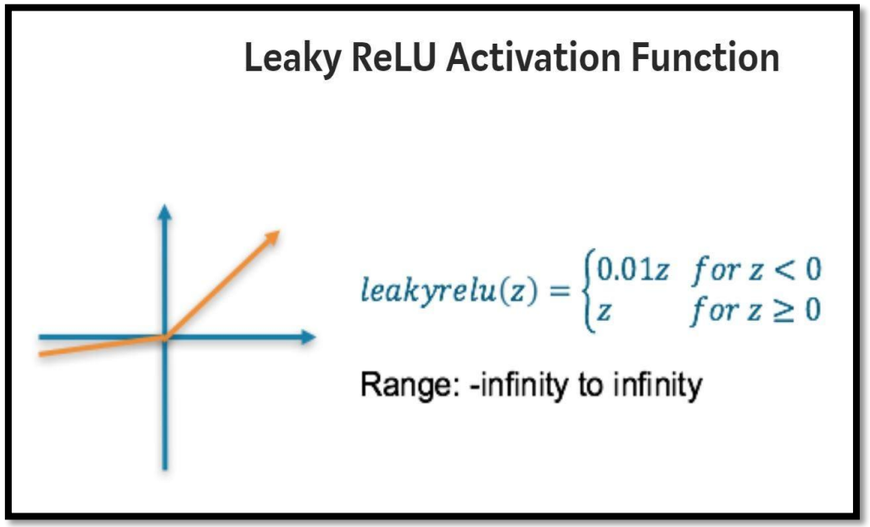
- Its notable efficiency makes it especially advantageous in **Convolutional Neural Networks (CNNs)** and other deep learning models, leading to its **adoption in advanced object detection frameworks such as YOLO** (You Only Look Once) **etc.**

Nature :- It is non-linear, which means we can easily **back propagate the errors** and have **multiple layers of neurons** being

i) Leaky ReLU Activation Function: Leaky ReLU function is nothing but an **improved version of the ReLU function**. As we saw that for the ReLU function, the **gradient is 0 for x<0**, which **would deactivate the neurons in that region.**

Leaky ReLU is defined to **address this problem**. Instead of **defining the Relu function as 0 for negative values of x,** we define it as **an extremely small linear component of x**

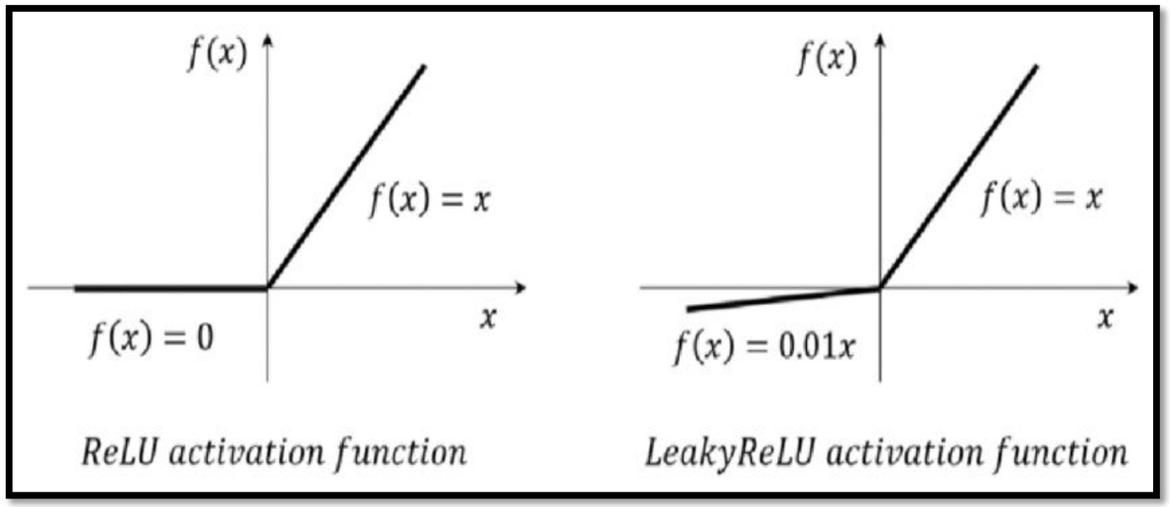
Here is the **mathematical expression-**



Can you see the Leak?

The **leak helps to increase** the **range of the ReLU function**. Usually, the value of **x is 0.01** or so.

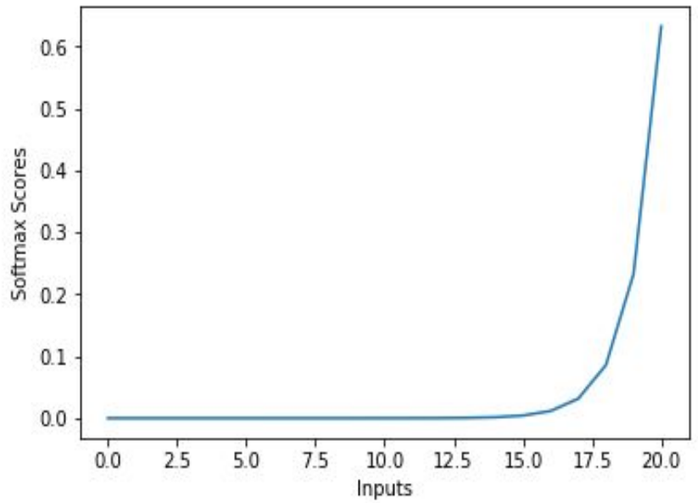
Therefore the **range** of the **Leaky ReLU is (-infinity to infinity).**



(D) Softmax Activation Function:

The softmax function is **more popular** for performing **multiclass classification tasks**.

- It simply takes **a vector of numbers as inputs** and produces a**nother vector containing the probability distribution of inputs**.



The softmax function is **also a type of sigmoid function** but is handy when we are trying to **handle multi- class classification problems.**

Uses :- Usually used when trying to handle multiple classes.

- The softmax function was commonly found in the output layer of **image classification problems**.
- **Softmax function** is often described **as a combination of multiple sigmoid.** We know that **sigmoid returns values between 0 and 1**, which can be treated as probabilities of a data point belonging to **a particular class.**

Here is the **mathematical expression-**

$$\sigma(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^{K} e^{z_k}} \quad \text{for } j = 1, \ldots, K.$$

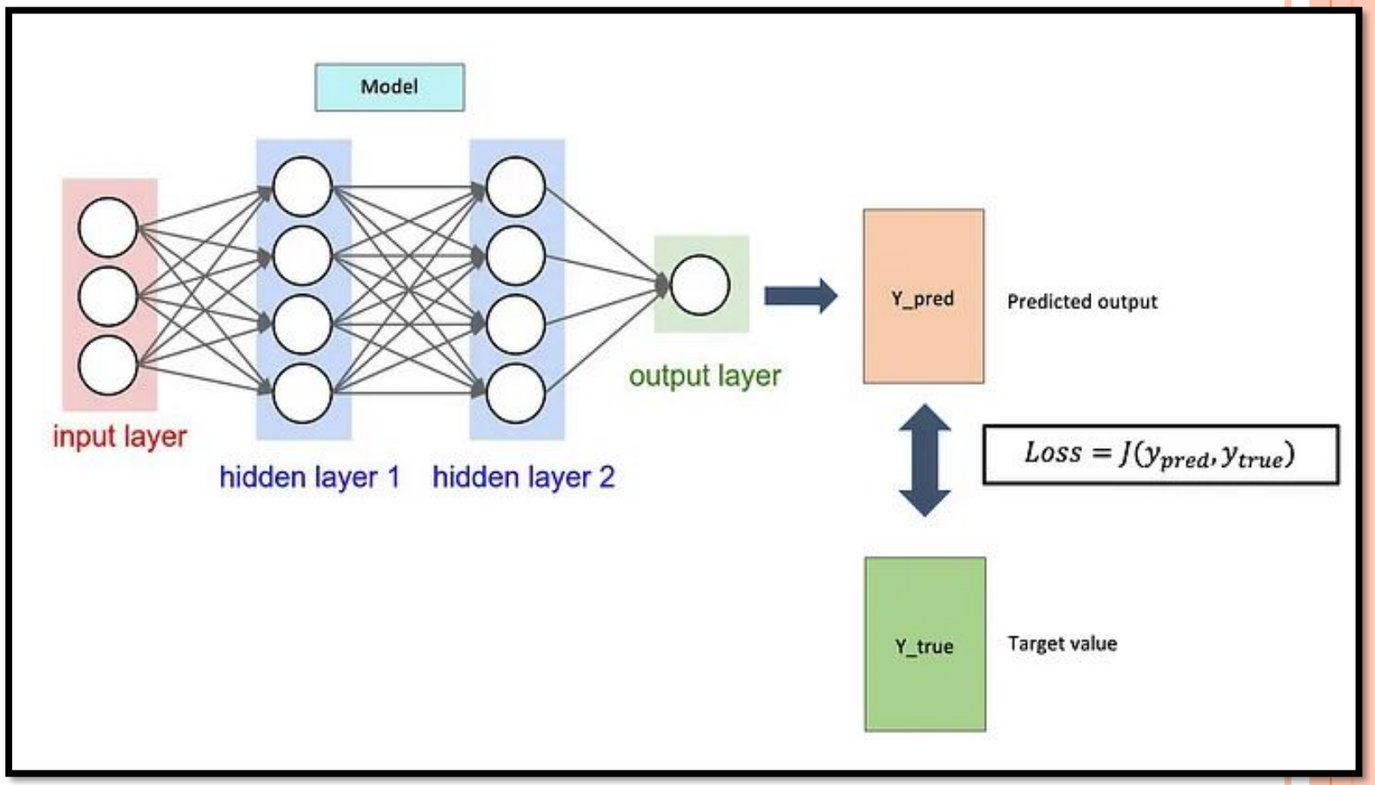
- While building a **network for a multiclass problem**, the output layer would have as **many neurons as the number of classes in the target**.
- For instance if **you have three classes**, there would be **three neurons in the output layer.** Suppose you **got the output from the neurons as [1.2 , 0.9 , 0.75].**

- Applying the **softmax function over these values**, you will get the following result – **[0.42 , 0.31, 0.27].** These represent the probability for the **data point belonging to each class.**
- Note that the **sum of all the values is 1.**
- . For instance, if you want to **classify images of dogs, cats, and, rabbits**, **Softmax is the best option.**

4. Loss Function

- A **loss function(or) Error Function** is a **mathematical function** that measures the **difference between the predicted output of the model and the actual output.**
- It **quantifies** how **well the model is performing on a single training example**.
- The **goal** of the deep learning model is to **minimize this loss function,** indicating a b**etter fit of the model** to the data.

- Let's define a **loss function (J)** that takes the following **two parameters**
  - **Predicted output (y_pred)**
  - **Target value (y_true)**



Types of Loss Functions:

1. **Regression Loss Function**
   a) **Mean Square Error(MSE)**
   b) **Mean Absolute Error(MAE)**
   c) **Root Mean Square Error(RMSE)**
2. **Classification Loss Function**
   i) **Binary Cross Entropy(Log Loss)**
   ii) **Categorical Cross Entropy**

1. Regression Loss Functions: Regression models deals with **predicting a continuous value .**

For Eg: Given floor area, number of rooms, size of rooms, predict the price of the room. The loss function used in the regression problem is called **"Regression Loss Function".**

1. Mean Squared Error/Squared loss/ L2 loss: The Mean Squared Error (MSE) is a **straightforward and widely used loss function.**

To calculate the MSE, you take **the difference between the actual value and the model prediction**, **square it**, and then **average it across the entire dataset**.

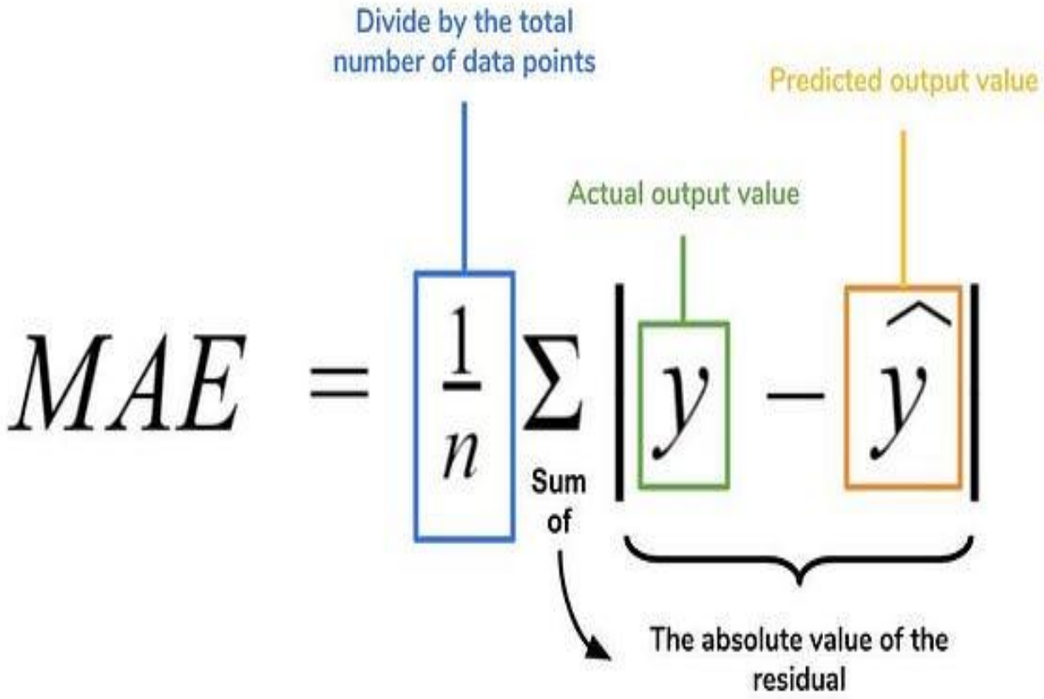
$$MSE = \frac{1}{N}\sum_{i}^{N}(Yi - \hat{Y}i)^2$$

Example:

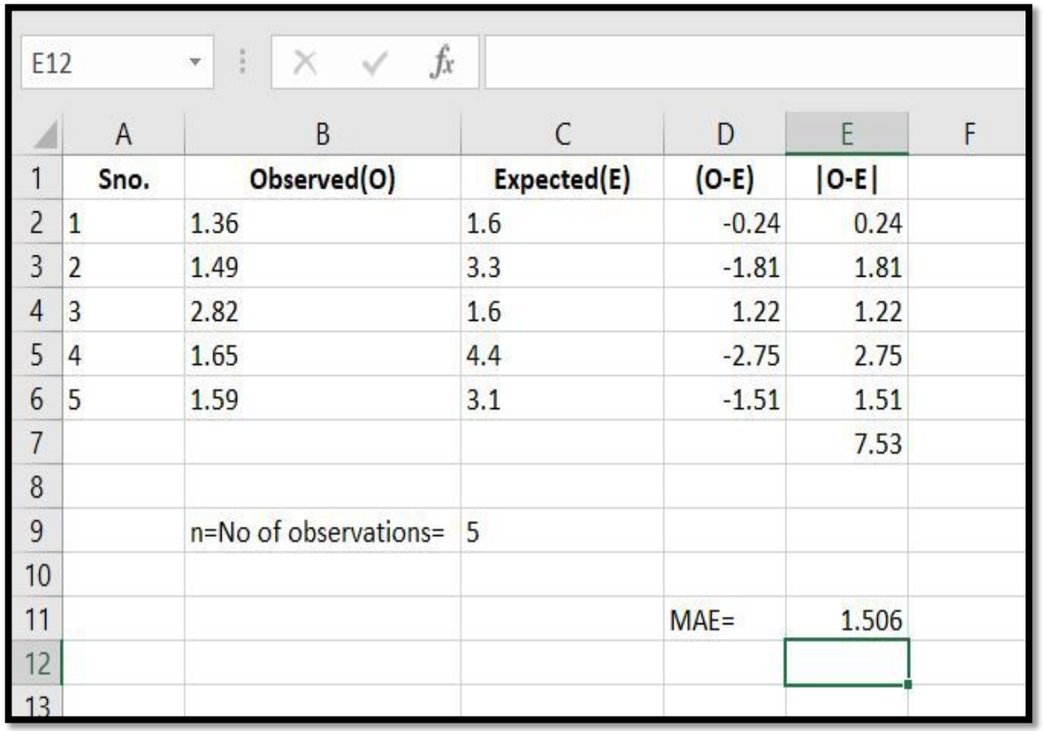
	A	B	C	D	E
1	Sno.	Observed(O)	Expected(E)	(O-E)	(O-E)^2
2	1	1.36	1.6	-0.24	0.0576
3	2	1.49	3.3	-1.81	3.2761
4	3	2.82	1.6	1.22	1.4884
5	4	1.65	4.4	-2.75	7.5625
6	5	1.59	3.1	-1.51	2.2801
7					14.6647
8					
9		n=No of observations=	5		
10					
11				MSE=	2.93294

Applications: Time Series Analysis, Student Academic Performance,

2. Mean Absolute Error/ L1 loss Functions: The Mean Absolute Error (MAE) is another **simple loss function**. It calculates the **average absolute** difference between the **actual value and the model prediction across the dataset.**



Example:



	A	B	C	D	E	F
1	Sno.	Observed(O)	Expected(E)	(O-E)	|O-E|	
2	1	1.36	1.6	-0.24	0.24	
3	2	1.49	3.3	-1.81	1.81	
4	3	2.82	1.6	1.22	1.22	
5	4	1.65	4.4	-2.75	2.75	
6	5	1.59	3.1	-1.51	1.51	
7					7.53	
8						
9		n=No of observations=	5			
10						
11				MAE=	1.506	
12						
13						

Case Studies:

1. A Cloth Retailer
2. A pharmaceutical company
3. An e-commerce platform

3. Root Mean Square Error(RSME): Root mean square error or **root mean square deviation** is one of the most commonly used **measures for evaluating the quality of predictions.**

$$RMSE = \sqrt{\frac{\sum_{i=1}^{N} (Predicted_i - Actual_i)^2}{N}}$$

$$RMSE = \sqrt{\frac{\sum_{i=1}^{n} (y_i - \hat{y}_i)^2}{n}}$$

Where **N is the number of data points,**

y(i) is the **i-th measurement**, and

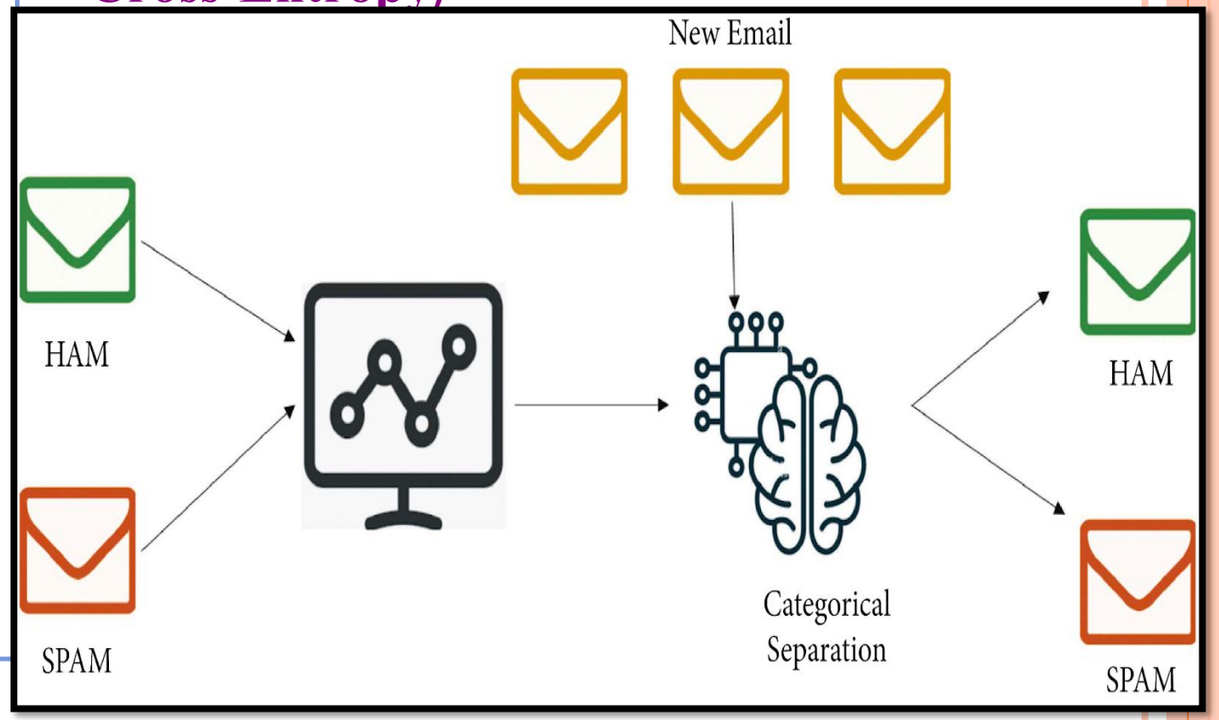
y^(i) is its **corresponding prediction**.

2. Classification Loss Functions:

i) Binary Cross Entropy(Log Loss): It is one of the most **common loss functions** used for **binary classification problems.**

It's suitable when dealing with **probability outputs.** For each instance, it penalizes the model **proportionally to the distance between the predicted probability** and the **actual label.**

Email Spam Detection (Binary Cross-Entropy)



$$\text{Log Loss} = -\frac{1}{N}\sum_{i=1}^{N} y_i \log \hat{y}_i + (1-y_i)\log(1-\hat{y}_i)$$

- yi – actual values
- yihat – Neural Network prediction

ii) Categorical Cross Entropy: Categorical Cross entropy is used for **Multiclass classification and softmax regression.**

$$\text{Loss} = -\sum_{j=1}^{K} y_j \log(\hat{y}_j)$$

where k is number of classes in the data

where
k is classes,
y = actual value
yhat – Neural Network prediction

- **It finds applications** in **various domains**, including **image classification, natural language processing**, **speech recognition**, **computer vision**, and **recommendation systems.**
- In **image classification**, it helps the model assign the **correct label to an input image.**
- While in **natural language processing**, it enables accurate **text classification or generation.**
- For **speech recognition**, it aids in **transcribing spoken words.**

5. Hyper parameters

- **Hyperparameters** are those **parameters** that are **explicitly defined by the user** to **control the learning process.**
- **Hyperparameters** determine **key features** such as **model architecture**, **learning rate**, and **model complexity.**
- Examples of hyperparameters include the **number of nodes** and **layers in a neural network** and the **number of branches in a decision tree**.

The Main Key points used in Hyperparameters are:

- Main parameters of the NN is **W and b**
- Hyper parameters (parameters that control the algorithm) are like:
  - **Learning rate.**
  - Number of **iterations.**
  - Number of **hidden layers L.**
  - Number of **hidden units n**.
  - Choice of ***activation functions***.
  - Batch and Epoch

1. Epoch: Represents **one iteration over the entire dataset** (everything put into the training model).

- An **epoch** completes **once a whole dataset** has undergone **forward propagation and back propagation.**

2. Batch – An epoch is made up of **batches**. Sometimes the whole dataset can not be passed through the neural network at once **due to insufficient memory or the dataset being too large.**

- We divide the entire dataset into **smaller numbers of parts called batches.** These batches are **passed through the model for training.**

3. Iterations: The **total number of batches** needed to **complete one epoch** is called **iteration**.

For example, the **dataset consists of 1000 images**. We divide **it into ten batches of size 100** each.

One iteration is completed when one batch passes through a neural network that is **forward propagation and back propagation.**

The **following equation** shows the relation between **batch size and iteration** with **epoch and can be used to calculate epoch size:**

$$epoch_size = batch_size \times iterations$$

Eg: Let's have the **training dataset having 1000 training samples**. And we want to **break the dataset into a batch size of 100**. Suppose we are **going for 5 epochs**, Then the **total number of iterations will be :**

Ans:

Total number of training samples = **1000**

Batch size **= 100**

Total number of iterations=Total number of training samples/Batch

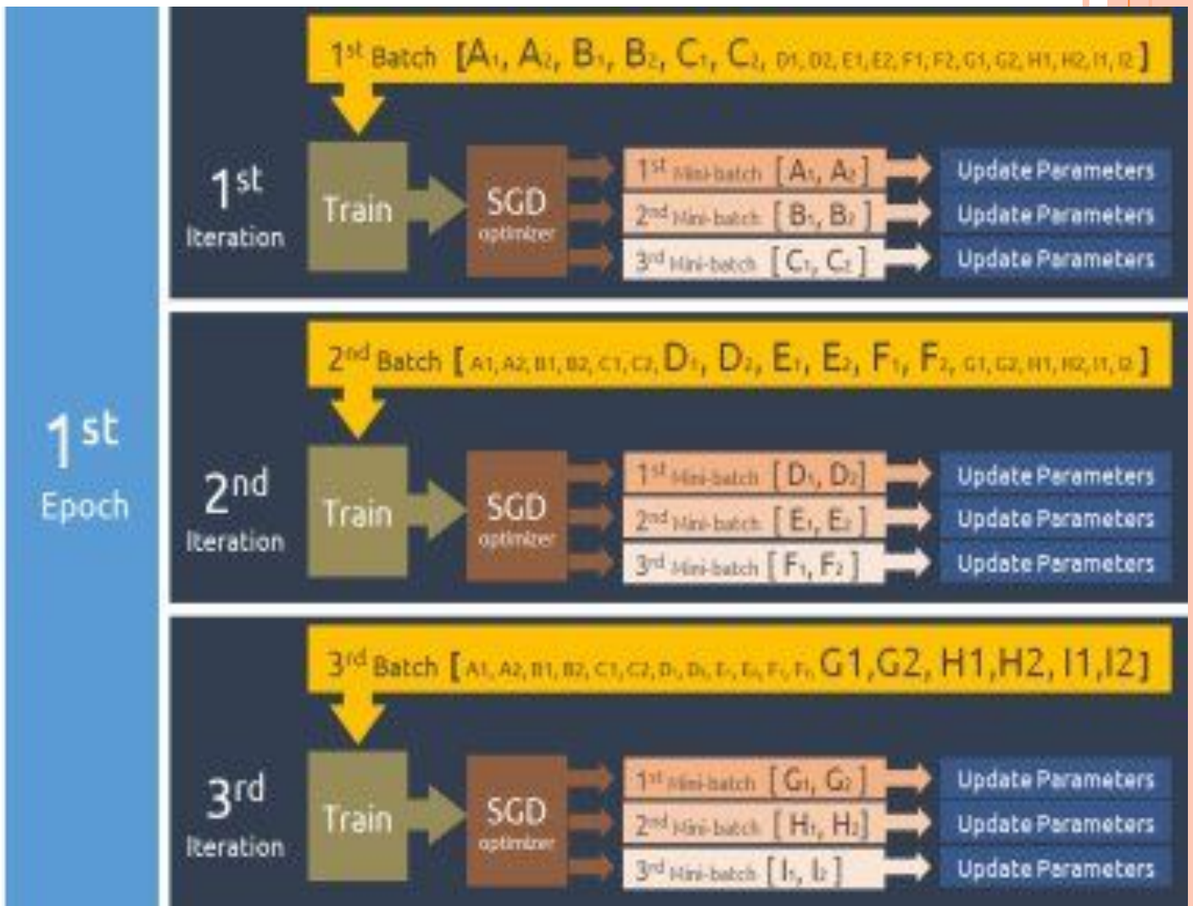
size=**1000/100=10**

Total number of iterations = **10**

One epoch = **10 iterations**

Total number of iterations in 5 epochs = **10*5 = 50 iterations**.

The following illustration can help in understanding the difference between the above terminologies:



1. **Learning Rate:** It is a Hyperparameter that provides the model a scale of how much model **weights should be updated, to minimize the Loss Function.**
2. **Optimizer:** In deep learning, optimizers are crucial as algorithms that **dynamically adjust the model's parameters** to **minimize the loss function**.

- There are **various optimization techniques** to **change model weights and learning rates,** like **AdaGrad, RMSProp, AdaDelta, and Adam.**

- **For eg :**
- **Adagrad** stands for **Adaptive Gradient Optimizer.** It is used to **reduce the loss function** with **respect to the weights.** The **weight updating formula is as follows**:
- Based $(w)_{new} = (w)_{old} - \eta \frac{\partial L}{dw(\ old\ )}$ **can be written as:**

$$w_t = w_{t-1} - \eta \frac{\partial L}{\partial w(t-1)}$$

where
w(t) = value of w at **current iteration,**
w(t-1) = value of w at **previous iteration and**
η = **learning rate.**

1. **Learning Rate:** It is a Hyperparameter that provides the model a scale of how much model **weights should be updated, to minimize the Loss Function.**
2. **Optimizer:** In deep learning, optimizers are crucial as algorithms that **dynamically adjust the model's parameters** to **minimize the loss function**.

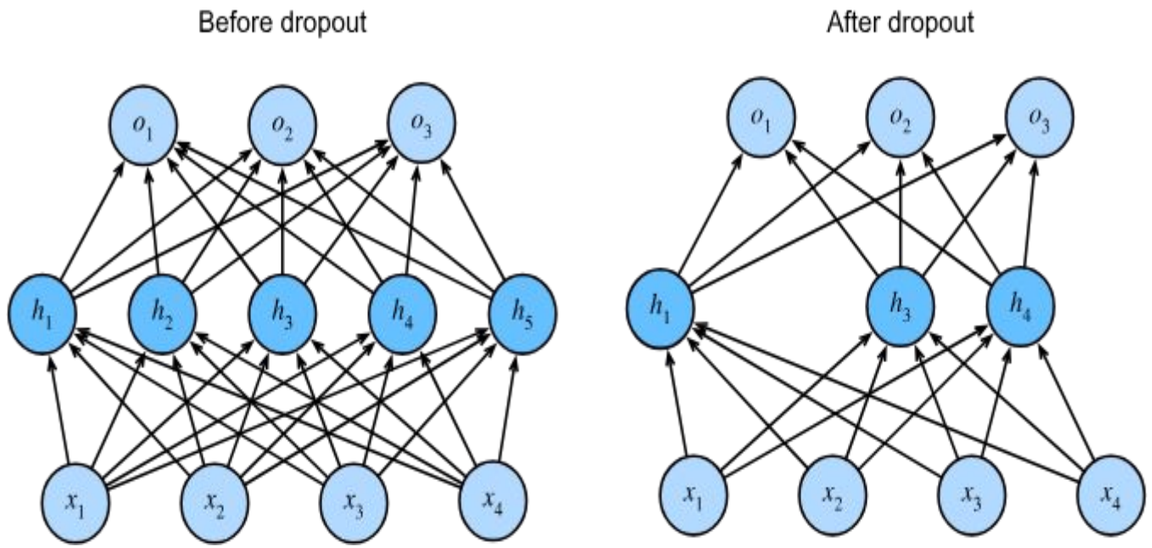
- There are **various optimization techniques** to **change model weights and learning rates,** like **AdaGrad, RMSProp, AdaDelta, and Adam.**

- **For eg :**
- **Adagrad** stands for **Adaptive Gradient Optimizer.** It is used to **reduce the loss function** with **respect to the weights.** The **weight updating formula is as follows**:
- Based $(w)_{new} = (w)_{old} - \eta \frac{\partial L}{dw(\ old\ )}$ **can be written as:**

$$w_t = w_{t-1} - \eta \frac{\partial L}{\partial w(t-1)}$$

where
w(t) = value of w at **current iteration,**
w(t-1) = value of w at **previous iteration and**
η = **learning rate.**

3. Dropout: Dropout was **introduced by "Hinton et al"** and this method is now very popular. It consists of **setting to zero** the **output of each hidden neuron in chosen layer** with **some probability** and is proven to be **very effective in reducing overfitting.**



- To achieve **dropout regularization**, some **neurons in the artificial neural network are randomly disabled.** That prevents them from being **too dependent on one another as they learn the correlations.**
- Dropout is driven by **randomly dropping a neuron** so that it will **not contribute to the forward pass and back-propagation.**

Thank You!

Common Architectural Principles of Deep Learning

Thank you for your attention!

Any Questions?

Topics Covered

- Common Architectural Principles of Deep Networks
- Parameters
  - 1. Weights
  - 2. Bias
- Layers
- Activation Functions
- Loss Functions
- Hyper Parameters

